Recursive WITH

How to find all cities in USA?

```
CREATE TABLE Area (
    area_id INT,
    area_name TEXT,
    area_type TEXT,
    area_parent INT,
    PRIMARY KEY (area_id),
    FOREIGN KEY (area_parent)
        REFERENCES Area (area_id));
```

```
INSERT INTO Area
VALUES
    (1, 'Earth', 'planet', NULL),
    (2, 'U.S.A.', 'country', 1),
    (3, 'Washington, D.C.', 'city', 2),
    (4, 'Texas', 'state', 2),
    (5, 'Harris', 'county', 4),
    (6, 'Houston', 'city', 5),
    (7, 'France', 'country', 1),
    (8, 'Ile-de-France', 'region', 7),
    (9, 'Paris', 'department', 8),
    (10, 'Paris', 'city', 9);
```

Hierarchy distance between country and city varies.

Solution outline

1. Find all “areas” that are in the U.S.
   a. Start with ‘U.S.A.’
   b. Repeatedly, include all areas inside a known U.S. area.

2. Filter those areas for those that are cities.

Areas computed recursively – recursive CTE

```
WITH RECURSIVE USAArea
    (area_id, area_name, area_type, area_parent)
AS
   (SELECT area_id, area_name, area_type, area_parent
    FROM Area
    WHERE area_name = 'U.S.A.'

    UNION

    SELECT a.area_id, a.area_name, a.area_type, a.area_parent
    FROM Area AS a
    INNER JOIN USAArea AS ua ON a.area_parent = ua.area_id)

SELECT area_id, area_name
FROM USAArea
WHERE area_type = 'city';
```

Base case

Must combine with UNION [ALL]

Inductive case

Evaluation example

WITH:

1. Base case selects 'U.S.A.' – id 2.
2. Union with inductive case selects children of just-added data – ids 3,4.
3. Union with inductive case selects children of just-added data – id 5.
4. Union with inductive case selects children of just-added data – id 6.
5. Union with inductive case selects children of just-added data – empty. **Induction terminates!**

SELECT:

1. Selects results with type 'city' – ids 3,6.

```
INSERT INTO Area
VALUES
    (1, 'Earth', 'planet', NULL),
    (2, 'U.S.A.', 'country', 1),
    (3, 'Washington, D.C.', 'city', 2),
    (4, 'Texas', 'state', 2),
    (5, 'Harris', 'county', 4),
    (6, 'Houston', 'city', 5),
    (7, 'France', 'country', 1),
    (8, 'Ile-de-France', 'region', 7),
    (9, 'Paris', 'department', 8),
    (10, 'Paris', 'city', 9);
```

Fibonnaci example

```
WITH RECURSIVE fib (num, f1, f2) AS
    (SELECT 0, 0, 1

     UNION ALL

     SELECT num+1, f2, f1+f2 FROM fib
     WHERE num < 20)

SELECT num, f1 AS fib_num
FROM fib;
```

Some details

- Common usage cases?
- Recursive CTEs restrict allowable queries. Restrictions vary.
  - PostgreSQL: One or more base cases. Listed before the one inductive case.
  - PostgreSQL: One use of recursion within the CTE.
- Cost of removing duplicates (UNION) might be higher than duplicating work (UNION ALL).